



COCOSSim: A Cycle-Accurate Simulator for Heterogeneous Systolic Array Architectures

Mansi Choudhary*

Dept. of ECE
Duke University
Durham, NC, USA
mansi.choudhary@duke.edu

Chris Kjellqvist*

Dept. of Computer Science
Duke University
Durham, NC, USA
cmk91@duke.edu

Jiaao Ma

Dept. of Computer Science
Duke University
Durham, NC, USA
jm790@duke.edu

Lisa Wu Wills

Dept. of Computer Science
Duke University
Durham, NC, USA
lisa@cs.duke.edu

Abstract—Performance modeling is an essential tool for enabling cost-effective and efficient exploration of architectural and microarchitectural design decisions for hardware development. However, existing simulators face limitations in architectural flexibility, memory hierarchy modeling, and support for modern accelerators that cater to the computational demands of contemporary machine learning models, where non-linear operations like softmax and other vectorized activations have become increasingly important.

To address these gaps, we present COCOSSim, a cycle-accurate performance simulator designed for evaluating architectural and microarchitectural modifications in heterogeneous systolic array-based accelerators. COCOSSim supports a wide range of neural network inference workloads, integrating systolic arrays with vector units for non-linear operations and DRAM-Sim3 for memory modeling. By offering a PyTorch frontend for seamless model integration, flexibility in scheduling strategies, and the ability to make architectural and microarchitectural modifications, COCOSSim enables detailed performance analysis and design exploration.

When validated against the Google TPU v3, COCOSSim achieves an average error rate of 13%. Additionally, COCOSSim is highly scalable and offers simulation speeds an order of magnitude faster than prior work while providing significantly greater flexibility and enhanced modeling capabilities. We present two case studies demonstrating how COCOSSim can be used to evaluate architectural modifications to modern heterogeneous accelerators: model parallelism and operation fusion.

Index Terms—Simulation software, Systolic arrays, Vector processing, Modeling, Neural network hardware

I. INTRODUCTION

In the rapidly evolving landscape of hardware design for neural network accelerators, performance modeling has become an essential tool for both researchers and industry professionals. It provides a cost-effective and quick way to evaluate performance, especially when developing heterogeneous, application-specific hardware. These models enable designers to predict performance outcomes early in the design process, allowing the designer to explore the effects of structural modifications and optimizations while bypassing the extensive time and financial investments required for RTL design, verification, and physical prototyping. This proactive approach not only accelerates innovation but also ensures that

* Both authors contributed equally to this research.

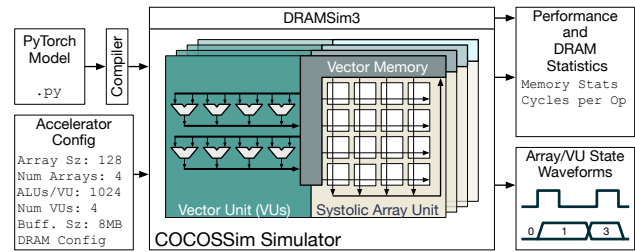


Fig. 1: COCOSSim models the execution of Neural Network models on an Accelerator Architecture. The model input can be provided in COCOSSim’s internal format or directly from PyTorch using our input compiler. COCOSSim models systolic arrays as well as vector units for Activations, Softmax, and other miscellaneous vector functions. COCOSSim outputs performance statistics, cycle-accurate state changes for each unit, as well as DRAM usage statistics from DRAMSim3 [16].

the final hardware solutions are high-performing and cost-effective.

While such modeling is crucial to justify and ensure the success of hardware accelerator design, it is burdensome to build bespoke modeling frameworks for each new architecture. For this reason, many prior works propose re-usable accelerator modeling frameworks with differing levels of architecture-specificity and flexibility. MAESTRO [15], for instance, proposes a highly flexible framework for Convolutional Neural Network (CNN) accelerator modeling that makes very few assumptions about the underlying architecture. Users provide accelerator architectures using high-level, software-pragmatic dataflow descriptors, and the modeling framework uses this description to infer performance. However, when the user wants to simulate specific, architectural behaviors (e.g., latencies for specific operations), such a high-level framework is inflexible to these desired modifications.

On the other end of the flexibility spectrum are cycle-accurate simulators for modeling existing architectures, such as DNNsSim [22], Timeloop [18], [30], STONNE [17], SCALE-Sim [26] or TPTPUSim [5]. TPTPUSim is a modeling framework for the Tensor Processing Unit (TPU), a commercial neural network training and inference engine from Google

that is now on its fifth generation [2], [8], [11]–[13]. While the TPU is closed-source and therefore impossible to model perfectly, TPTPUSim models with reasonable accuracy the performance characteristics of various on-chip components, such as the interconnect latencies. Using this framework, the user can explore basic modifications to the TPU parameterization, but it is challenging to make substantial hardware modifications.

Recent state-of-the-art models, such as generative large language models [4], [6], [27], [28] or generative image models [7], [10], [20], [24], [25], are growing in size and complexity, necessitating modifications in hardware architectures to efficiently support their training and inference. One key modification is the inclusion of vector units. Existing simulators were developed when CNNs were the primary workload. CNN runtime is dominated by large matrix-multiply operations, which are performed by systolic arrays. Accordingly, these existing frameworks primarily model systolic arrays. In contrast, recent generative models consist of both matrix multiplies and softmax — a long-latency, memory-intensive operation poorly suited to systolic arrays.

Practical systolic array accelerators typically feature *heterogeneous* and *concurrent* compute to address the computational challenges of neural network training and inference. Heterogeneous architectures contain different hardware dataflows for computing different parts of the neural network. Concurrent architectures consist of multiple identical compute units, usually subdividing large operations into smaller operations and executing them in parallel. The TPU is heterogeneous and concurrent, using multiple vector units for performing softmax, data movement engines for performing matrix transposes, and systolic arrays for matrix multiplies.

Evaluating this heterogeneity and concurrency is challenging using existing methodologies. High-level modeling methodologies can quickly produce performance inferences but may elide microarchitectural latencies crucial for accurately measuring and diagnosing a particular configuration’s benefits or drawbacks. On the other hand, low-level, cycle-accurate methodologies provide accurate models that provide knobs for a variety of details but are inflexible to significant architectural or microarchitectural changes that the original authors did not anticipate. For instance, how does performance change if we can utilize vector units and systolic arrays in parallel to execute separate neural network layers? Existing methodologies [15], [17], [18], [26] are explicitly sequential, making it challenging to answer such a question.

In this work, we propose COCOSSim¹, a flexible, cycle-accurate performance simulator for evaluating both architectural and microarchitectural modifications to systolic-array-based accelerators on arbitrary neural network inference workloads, shown in Figure 1. Taking a PyTorch [19] model and an accelerator configuration as input, COCOSSim maps PyTorch operations to a job graph for the provided hardware dataflow.

¹COCOSSim stands for a Cycle-accurate, Offload, Configurabe, and Open-source Systolic-array-based accelerator Simulator.

Then it models the execution of this job graph, providing performance metrics, memory statistics, and cycle-accurate waveform of the state machine for each hardware dataflow. COCOSSim is integrated with DRAMSim3 [16], a cycle-accurate DRAM modeling framework, allowing the user to model the impact of external memory configurations on model execution. COCOSSim currently supports configurations for Weight-Stationary (TPU-like), Output-Stationary, and vector unit dataflows with support for concurrent execution of multiple systolic arrays and vector units. We provide a scheduler that we believe aligns well with the TPU v3 scheduler. We provide each of these components and dataflows with the intent to be modifiable, allowing for explorations on every facet of model execution: scheduling, architecture (e.g., changing the number of systolic arrays or vector units), and microarchitecture. We make the following contributions:

- COCOSSim, a fast and cycle-accurate open-source simulation framework for performance modeling of concurrent, heterogeneous systolic array architectures.
- We provide a PyTorch frontend that compiles neural network models into the COCOSSim input format. Users can also provide inputs directly without using this compiler.
- We evaluate modeling accuracy and simulation performance of COCOSSim against TPTPUSim and SCALE-Sim on a variety of different neural network layers. Our results demonstrate that COCOSSim achieves 60% lower error rate and significantly higher simulation speed compared to prior work. We present two case studies that explore how COCOSSim can be used to experiment with modern techniques in machine learning acceleration: concurrent heterogeneous accelerators and operation fusion.

II. BACKGROUND

A. Systolic Arrays

Systolic arrays have emerged as a key architectural paradigm for hardware acceleration of machine learning workloads due to their efficient handling of matrix multiplication. These arrays consist of a grid of processing elements (PEs) that execute simple operations, typically multiply-and-accumulate (MAC), and pass intermediate results to neighboring PEs. This pipelined design leverages machine learning workloads’ inherent parallelism and data locality, enabling high throughput and energy efficiency.

In a typical systolic array setup, matrix multiplication is expressed as $M \times K \times N$, where the input activation matrix of dimensions is $M \times K$, the weight matrix of dimension is $K \times N$, and the output activation matrix dimension is $M \times N$.

One of the key advantages of systolic arrays is their ability to minimize memory access by reusing data locally within the array, reducing the energy and latency penalties associated with external memory transfers. To optimize performance for different workloads, systolic arrays support various dataflow strategies:

- Output-Stationary (OS): In this approach, the output partial sums are reused by storing them in registers inside the Processing Elements (PEs).

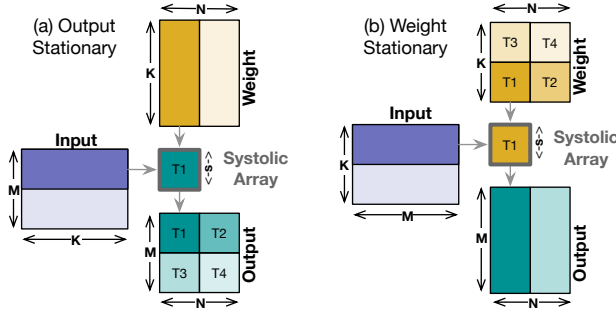


Fig. 2: Comparison of Systolic Array Dataflow Approaches: (a) **Output-Stationary (OS)**: The output matrix is divided into tiles that remain stationary within the systolic array. The input activation and weight matrices stream through the array, and partial sums accumulate in the systolic array. This minimizes data movement for outputs, maximizing reuse when M and N are small relative to K . (b) **Weight-Stationary (WS)**: The weight matrix tile is prefilled into the systolic array. The input activations stream through the systolic array and output activation partial sums are streamed out. This strategy maximizes reuse when the weight dimensions K and N are small relative to M . The tiling in both approaches ensures maximum reuse of the buffered input activations.

- **Weight-Stationary (WS)**: In this approach, weights are kept stationary within PEs, allowing for efficient reuse across multiple input activations.

Because the dimension of the systolic array, s , is typically smaller than the dimensions of the input/output matrices, the matrix multiply is tiled into smaller matrix multiplies. While tiling enables systolic arrays to perform large matrix multiplications, it requires reading the same operands from memory multiple times. For this reason, systolic array architectures are outfitted with on-chip SRAM buffers to store slices of the input matrices for reuse. The two dataflow strategies and their optimum tiling strategies are described in Figure 2.

B. Tensor Processing Unit (TPU)

TPU is a commercial hardware accelerator based on weight-stationary systolic arrays designed and optimized for machine learning workloads. Although the first iteration of the TPU only contained systolic arrays focusing on accelerating dense matrix-multiply operations, future iterations have become increasingly heterogeneous, adding units to support the other operations that have become common in machine learning workloads. For instance, recent TPUs have added Vector Processing Units (VPUs), data movement engines for computing matrix transposes and permutations, and scalar units for control flow. TPUs also have large on-chip SRAM buffers for storing portions of the input matrices. While plain systolic arrays can be easy-to-model architectures with clear trade-offs, the heterogeneity of modern accelerators makes them much more challenging to model and reason about.

III. COCOSSIM'S DESIGN

COCOSSim provides a simulator infrastructure for modeling architectural, microarchitectural, and scheduling modifications for neural network inference accelerators.

While existing simulator architectures focus primarily on the number of accumulators, the dataflow structure and the penalties that follow from that structure are important for performance prediction. For instance, systolic arrays pay a "drain" penalty proportional to their size. Whether this systolic array is weight-stationary, output-stationary, or something else determines *when* and *how often* this penalty is paid. Other latencies, such as FPU latency or DRAM access latency, may interact with the structure of the accelerator in hard-to-predict ways. As such, it is important that a simulator can incorporate these latencies to improve prediction accuracy.

A. Modeling Methodology

Our simulator is structured into three primary components. First, compute units are modeled as state machines. We chose state machines because they are simple, computationally efficient, and can express behaviors at whatever level of abstraction the programmer requires. Hardware that requires exceptional accuracy can modify and tune the state machine to its satisfaction, while simple compute models can be modeled by a similarly simple state machine. The following 7-tuple describes the state of a compute unit.

- 1) *state*: The current state in the hardware's control flow.
- 2) *job*: This contains any arbitrary metadata about the current operation being performed by the compute unit—for instance, the matrix dimensions for matrix multiplication.
- 3) C : the minimum number of cycles until transition. Even with infinite memory bandwidth, a datapath will take several cycles, determined by the datapath's latency and throughput, to compute an operation on the inputs, whose dimensions are specified by the current job.
- 4, 5) R, R' , the number of bytes to issue read requests for and receive responses for.
- 6, 7) W, W' , the number of bytes to issue write requests for and receive responses for.

On each cycle, C is decremented, and if R, W are greater than 0, memory requests are submitted to the memory controller. The memory controller, after completing a request, decrements R', W' . Once C, R, R', W, W' are all 0, the state machine transfers to the next stage of its execution as a function of *state* and *job*. The transition matrix specifies 3 parameters $C_{\text{next}}, R_{\text{next}}, W_{\text{next}}$. Upon transition, the corresponding parameters in the machine's state are set.

The second primary component of our simulator is the Memory Controller. State machines load memory requests on a global memory transaction queue, and, on each cycle, we enqueue up to one DRAM transaction per channel from the global queue to DRAMSim3 [16], a cycle-accurate DRAM simulator. Once a request to DRAM has been completed, the DRAM model decrements R', W' . Modeling R, W separately from R', W' ensures that we consider realistic rates for sending and receiving transactions to/from DRAM.

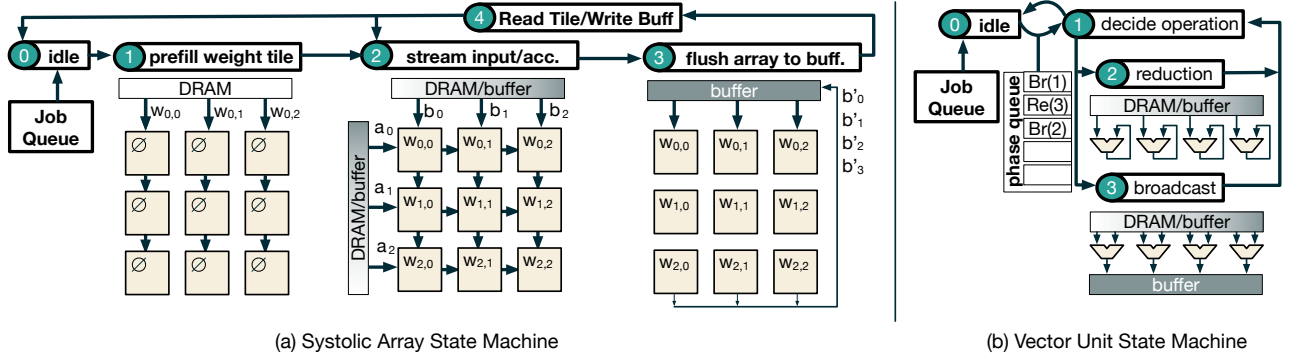


Fig. 3: Each unit is modeled as a state machine. (a) The Weight-Stationary Systolic Array state machine first loads a tile of the weight matrix into the array’s accumulators. In stage 2, the inputs, a , are multiplied with weights, w , and accumulated with b . The updated accumulators, b' , are stored in the on-chip buffer. Once each tile of the weight matrix has been processed, the accumulators are written to memory. (b) The Vector Unit supports an arbitrary number of passes over the input data. Each pass is either a “reduction” or “broadcast.” Reduction operations have a read-after-write dependency for each element in the vector, limiting the feasible parallelism for an operation. Broadcast operations can execute every element of the input vector in parallel.

State	C_{next}	R_{next}	W_{next}
0 $\rightarrow$ 1	s	$s^2 D$	0
1 $\rightarrow$ 2	$\max(l_{\text{fpu}}, B) * M$	$MDsB$	0
2 $\rightarrow$ 3	$\max(l_{\text{fpu}}, B) * s$	$s^2 D$	0
4 $\rightarrow$ 2	$\max(l_{\text{fpu}}, B) * M$	0	0
3 $\rightarrow$ 4	0	0 $MDsB$	0 $MNBD$

TABLE I: The state transition requirements for our weight-stationary systolic array model, shown in Figure 3(a). State transitions require that a certain number of cycles pass (C) and that a certain number of bytes are read(R_{next})/written(W_{next}) to memory before transitioning to a new state, as a function of systolic array dimension s , FPU latency l_{fpu} , data width (bytes) D , batch size B , and output matrix dimensions $M \times N$.

The TPU v3 uses High-Bandwidth Memory (HBM) stacks with a reported 900GB/s throughput [12], or 225GB/s per systolic array. During our benchmarks, we found that each channel of HBM is capable of approximately 25-29GB/s memory throughput. Accordingly, we configure DRAMSim to use an HBM2 module with eight channels per systolic array to achieve approximately the desired bandwidth.

The final component is the Job Scheduler, which we discuss in Section III-D.

B. Systolic Array Modeling

We specify the state transition matrix for our TPU-like weight-stationary systolic array in Table I and Figure 3(a). At a high level, weight-stationary systolic arrays function in the following way. ① They load a tile of weights into the systolic array. ② Input and output accumulations are streamed from memory or an on-chip buffer through the systolic array. ③ Once these operands have streamed in, the outputs must be fully flushed from the array into the output buffer. ④ The systolic array either transitions to the next tile in the job or if

there are no tiles left, writes the entire output matrix back to memory.

As we implement it, the state machine encodes architectural decisions that we believe the TPU makes based on our experimental results. For instance, we overlap the fetching of the weights for the next tile in the weight matrix while flushing the accumulations back to the output buffer. This hides the request’s latency, increasing utilization of the systolic array.

The transition matrix can specify the R_{next} , W_{next} parameters as a function of the current job progress. Figure 2 shows that the weight-stationary systolic array moves horizontally from Tile-1 to Tile-2. This permits it to re-use the previously loaded input matrix slice so long as the next tile is on the same row of tiles in the tile grid ($R_{\text{next}} = 0$). Otherwise, the array must read the next input matrix slice from memory ($R_{\text{next}} = MDsB$). Similarly, for W_{next} , we only need to write to memory if we have just computed the last tile in the weight matrix.

C. Vector Unit Modeling

Vector Units are responsible for computing various functions with differing computational patterns and complexities, so we sought a general way to express and analytically model vector dataflows. We model dataflows as consecutive phases of reduction and broadcast operations. A reduction operation is serialized over every element in a set, whereas a broadcast can be performed on every set element in parallel.

Given a vector of size P , we can specify two parameters that describe how a given dataflow can be performed for this set: P_{par} , P_{lin} such that $P = P_{\text{par}} * P_{\text{lin}}$. P_{lin} specifies the linear dimension of this vector: how many elements of a set take part in a reduction operation. P_{par} specifies the parallel dimension of the vector: how many independent sets are there in the input. For instance, summing each element of a vector of size 15

State	C_{next}	R_{next}	W_{next}
$1 \rightarrow 2$	$l_{\text{op}} * P_{\text{lin}} * \lceil P_{\text{par}} \div V \rceil$	0	PD
$1 \rightarrow 3$	$t_{\text{op}} * P \div V$	0	PD
$1 \rightarrow 0$	0	0	PD

TABLE II: The state transition requirements for our vector unit model, shown in Figure 2(b), specified by vector size P , data size (bytes) D , operation latency/throughput $l_{\text{op}}/t_{\text{op}}$, and number of vector ALUs V . Data is only read from DRAM in the first phase.

would have $P_{\text{par}} = 1, P_{\text{lin}} = 15$, whereas doing this for three independent vectors in parallel would have $P_{\text{par}} = 3, P_{\text{lin}} = 15$.

A Vector Unit job is specified as consecutive phases of reduction and broadcast operations. Each phase additionally provides a parameter l_{op} or t_{op} that specifies the latency of an operation for reduction, $\text{Re}(l_{\text{op}})$, or the throughput of a broadcast operation, $\text{Br}(t_{\text{op}})$. We use these primitives to construct complex vector operations. The vector unit state transition requirements are specified in Table II.

Softmax

Softmax is a performance bottleneck for language models that are not modeled by existing methodologies for systolic array accelerators.

$$\text{softmax}(\vec{x}) = \frac{\exp(\vec{x})}{\sum \exp(\vec{x})}$$

Softmax requires three passes. First, $\exp(\vec{x})$ is computed - a broadcast of the $\exp()$ operation, $\text{Br}(1)$. Second, we compute the sum of the prior operation - a reduction operation for which we select $l_{\text{op}} = 1, \text{Re}(1)$. And lastly, we use this sum to normalize $\exp(\vec{x})$ - a final broadcast, $\text{Br}(1)$. The input to softmax in language models is typically not a vector, but a matrix of size $M \times M$ where softmax is performed independently on each of the M rows. Accordingly, a softmax job is enqueued with $P_{\text{lin}} = P_{\text{par}} = M$.

Normalization

Vector units can also be used for normalization. A common normalization method is groupnorm, a modified version of layernorm.

$$\text{layernorm}(\vec{x}) = \frac{\vec{x} - \mathbb{E}[\vec{x}]}{\sqrt{\text{Var}[\vec{x}] + \epsilon}} * \gamma + \beta$$

We model this as the phase list: $\text{Re}(1), \text{Re}(4), \text{Br}(1)$, for the computation of the mean, variance, and final normalization, respectively. Groupnorm modifies layernorm by splitting input vector $\vec{x}$ into a number of groups, g , allowing the norm for each group to be performed in parallel. Accordingly, when the user provides an input with size $(P_{\text{par}}, P_{\text{lin}})$, we add it to the job queue as a layernorm operation with size $(P_{\text{par}} * g, P_{\text{lin}}/g)$.

Although it is impossible for us to know the specific dataflow and values of $l_{\text{op}}, t_{\text{op}}$ for these operations on commercial hardware, we tuned these values using our experimental data to approximate TPU v3 performance with reasonable accuracy.

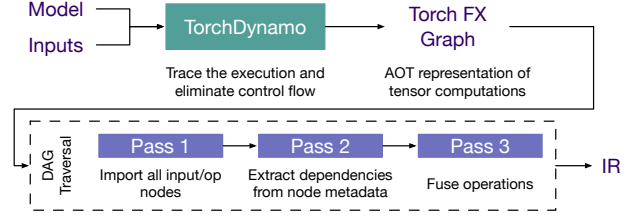


Fig. 4: Major components of the frontend of COCOSim, which captures and transpiles the model by leveraging TorchDynamo [3] to integrate with the PyTorch ecosystem.

D. Buffers and Job Scheduling

The third major component of our simulator is the job scheduler. Until now, we have assumed that the inputs to each operation, once initially read from DRAM, can be completely buffered in on-chip memory. In practice, this is not true.

Large operations must be tiled into smaller operations to utilize on-chip buffers appropriately. The strategy for this tiling is complex and has been a subject of prior research using dataflow-mapping methodologies [15], [18], [26] that map a software dataflow onto a hypothetical accelerator.

The job scheduler is responsible for splitting large jobs into jobs that fit in on-chip buffers in a way that can also utilize multiple cores. While there are many ways to split large jobs, we found it challenging to deduce the TPU’s job scheduling algorithm purely from performance results (profiling tools do not report core utilization). For instance, jobs that could utilize multiple cores sometimes do not use them. Nevertheless, we approximated the job scheduling algorithm of the TPU to the best of our ability.

The vector unit splits jobs to ensure that the linear dimension is never split between job tiles. That is, for buffer size S_{buff} , we find a P'_{par} such that $P'_{\text{par}} * P_{\text{lin}} \leq S_{\text{buff}}$. For systolic array jobs, we split along the N dimension because it maximizes the reuse of the output buffer.

E. Frontend for PyTorch Model Transpilation

COCOSim aims to provide seamless integration with the PyTorch ecosystem by offering a frontend that does not require additional modification to the user model prior to the simulation. The frontend is responsible for *capturing* the input dimensions and operation dependencies and *transpiling* the model into our in-house intermediate representation (IR) that the simulator can later use.

Figure 4 illustrates the major components of COCOSim frontend. It leverages TorchDynamo [3], a tracer to capture the execution of model inference. TorchDynamo produces a Torch FX graph [21], where all control flow has been unrolled and eliminated to allow static ahead-of-time (AOT) scheduling. With the sample inputs provided, the dimensions of each tensor can be determined. COCOSim then goes through three passes to traverse the DAG. During *Pass 1*, all nodes representing either an input tensor to the entire model or tensor operations are imported. *Pass 2* establishes

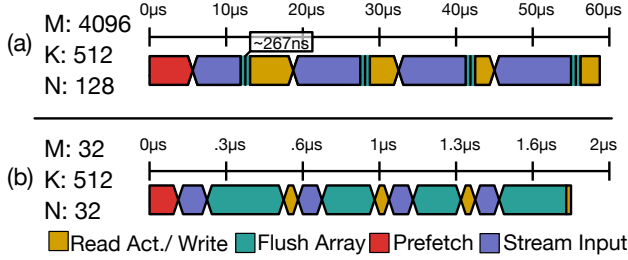


Fig. 5: COCOSSim provides waveforms that show the state of the systolic array throughout its execution, providing insights for optimization for the architecture on specific operations. (a) Waveform for a $4096 \times 512 \times 128$ matrix multiply. The penalties due to array flushing are marginal. (b) Waveform for a $32 \times 512 \times 32$ matrix multiply. The penalties due to array flushing constitute approximately two-thirds of the runtime.

the connectivity between nodes by extracting the Torch FX metadata. *Pass 3* performs pattern matching such as fused attention and softmax to allow simulation on fused operations. The frontend eventually produces an IR file for the job scheduler.

F. Output Statistics and VCD

COCOSSim provides output statistics in three main forms: high-level performance statistics, DRAM statistics, and cycle-accurate state waveforms. The high-level statistics include typical performance metrics: net runtime, time-per-operation, and average ALU utilization. DRAM statistics are provided through DRAMSim3, which provides an array of highly detailed statistics.

COCOSSim also supports dumping cycle-accurate state updates for each array into a Value Change Dump (VCD) waveform. In practice, we found that the waveform outputs were the most helpful tool for performance verification and characterization because they explain how the overheads encountered during execution result in the observed performance. Figure 5 illustrates how two matrix multiply waveforms may inform different design modifications for a 128×128 systolic array. Long input streams allow for good utilization of the array in Figure 5(a) with moderate overheads for memory access for fetching new input activations. In contrast, the short streams and small input matrices mean that array flush penalties are significant for the matrix multiply in Figure 5(b). Instead of buffering improvements, this motivates a smaller systolic array, a different type of systolic array, or a different architecture altogether. We present two case studies in Section V demonstrating how we can utilize these output modalities to explore opportunities for concurrent heterogeneous computations.

IV. EVALUATION

A. Simulation Speed

Table III compares the simulation time for different layer sizes across COCOSSim, TPTPUSim, and SCALE-Sim. As matrix sizes increase, these methodologies exhibit a significant

Size	COCOSSim	SCALESim	TPTPUSim	STONNE
128	14ms	0.42s	0.14s	8.73s
256	32ms	1.2s	0.79s	33.1s
512	0.12s	6.1s	7.96s	3.6min.
1024	0.59s	45s	113s	27min.

TABLE III: Matrix multiplication (matmul) simulation latencies on different frameworks. Size refers to the dimensions ($M = K = N$) for the matmul. For the largest matmul size, COCOSSim is almost two orders of magnitude faster than the next fastest simulator, SCALE-Sim.

rise in simulation time, making them unsuitable for large-scale simulations and design space explorations.

In contrast, COCOSSim demonstrates consistently fast simulation speeds across all tested layer sizes, achieving results on large matrix operations within seconds, tens to hundreds of times faster than competing methodologies.

B. Validation

We profiled the Stable Diffusion model [24] on Google TPU v3 using Tensorboard [1] to obtain layer-wise runtime metrics. We choose this model to validate our simulator due to its varied layer types and dimensions. The TPU v3 architecture was modeled within COCOSSim, SCALE-Sim, and TPTPUSim frameworks. We executed various linear and non-linear operations from the Diffusion model—such as general matrix multiplications, convolutions, softmax, and other activation functions—within the simulation frameworks and compared their performance metrics with the actual measurements obtained on the TPU v3.

Figure 6 shows the actual latencies of our selected workloads on TPU v3 and the latencies predicted by three simulators - COCOSSim, SCALE-Sim, and TPTPUSim. COCOSSim achieves lower average error (17%) than both SCALE-Sim (41%) and TPTPUSim (53%) for individual operations. We observe that COCOSSim incurs significant errors for some extremely low-latency operations. We hypothesize that these errors are caused by not modeling some of the startup overheads. Although our state machine modeling methodology can express these latencies, reliably measuring these nanosecond-scale latencies using our profiling methodology is difficult and presents significant noise. We chose not to pursue modeling these errors further as they are small enough that they only marginally affect our end-to-end results.

Figure 6 also compares COCOSSim’s predicted latencies for complete model layers against actual latencies recorded on TPU v3, achieving an average 13% error. As TPTPUSim and SCALE-Sim only the model matrix multiplies, we do not include latencies for softmax or normalization in these results for those simulators. This is consistent with the original methodologies [26].

During our validation, we repeatedly found that modifying our model parameters or state machines to model one operation more accurately often decreased the accuracy for another operation. We iteratively determined likely candidates for slowdowns by inspecting our waveform report and modify-

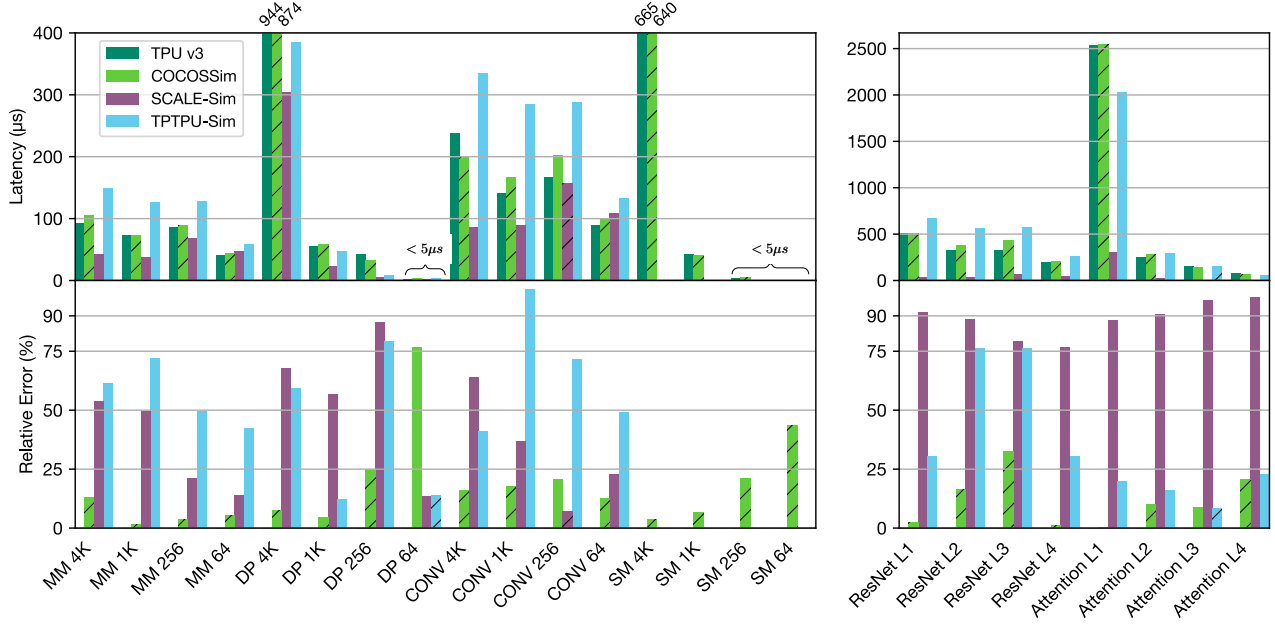


Fig. 6: Predicted latency and relative error of COCOSSim, SCALE-Sim, and TPTPU-Sim against the TPU v3 ground truth. The most accurate prediction is hatched. The chosen operations are a set of primitives for a range of sizes present in our models: Matrix-Multiply (MM), Multi-Headed Attention Dot-Product (DP), Convolution (CONV), SoftMax (SM). We evaluate neural network layers, ResNet and Attention, on the same set of simulators.

ing our state machine or microarchitectural parameters. After numerous iterations, we converged on a model that achieves the reported accuracy. Accurately modeling overheads is vital for identifying important opportunities.

V. CASE STUDIES

We present the following two case studies to show how COCOSSim supports new avenues for architectural exploration. Specifically, whereas prior simulation methodologies model sequentially scheduled, homogeneous accelerators, COCOSSim is capable of heterogeneous, parallel computing. We show that COCOSSim reveals and enables the analysis of these opportunities.

A. Model Parallelism

One drawback of heterogeneous architectures is the low utilization of highly specialized units, as they cannot effectively be used for all tasks. For instance, the systolic array remains idle while the TPU performs softmax using the vector units. While the TPU does not currently support simultaneous utilization of the vector unit, it is worth exploring how performance might improve if it could.

To see the potential benefits of such a scheme, we simulated the largest self-attention layer in our baseline Stable Diffusion [24] model. The different components of the layer are shown in Figure 7(a), where component 4 is the softmax operation. We made two observations. First, softmax comprises approximately 0.63ms (15%) of the runtime of the layer. If we

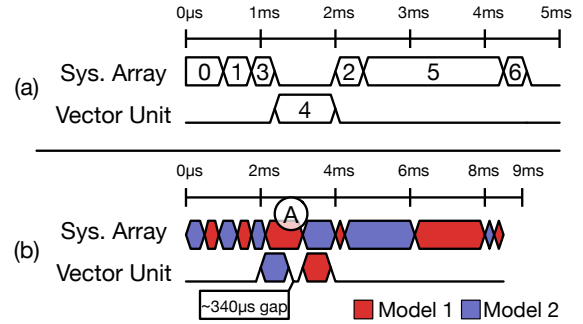


Fig. 7: (a) The systolic array is idle during the execution of softmax during self-attention. This presents an opportunity to hide the latency of softmax by overlapping its execution with matrix multiplies. Separate layers in the model execution are identified by number. (b) Using model parallelism to provide the aforementioned overlap only achieves a portion of the possible speedup — the additional memory intensity of executing softmax during systolic array execution slows down the systolic array.

can hide the latency of the softmax operation using other work, these savings would be significant. One common technique for achieving this is model parallelism, where separate inference tasks (not batches) are executed simultaneously using the same shared hardware.

COCOSSim can be used to dump arbitrary statistics to

the waveform. We added a log for a “bottlenecked by memory” flag, which is raised whenever the state machine has 0 required cycles remaining until transition but is prevented from transitioning because of outstanding reads/writes: $(C = 0) \wedge (R', W' > 0)$. This led us to our second observation: the systolic array is not memory-bound during its execution, whereas memory bandwidth is the bottleneck for the vector unit during software execution. This is good; if both units were memory-bound, running them in parallel would not improve performance.

With the expected speedup predicted and feasibility confirmed by our model, we went forward with running two models in parallel. The waveforms for this execution are shown in Figure 7(b). We scheduled jobs onto the hardware in a FIFO fashion, resulting in the alternating job schedule shown.

This change resulted in a total savings of 0.37ms, which is less than our predicted result of 0.63ms. This is because contention between hardware units during parallel execution slowed down the execution of the systolic array. Instead of using the *spare* memory bandwidth for performing softmax during the execution of matrix multiplies, softmax took the entire bandwidth for itself!

We can confirm this using waveform statistics provided by COCOSSim. The layer labeled ‘A’ in Figure 7(b) only takes a fraction of a millisecond before we enable model parallelism but balloons to nearly 1ms after. We also observe that whereas the “memory bottleneck” flag is almost always low before adding model parallelism, it is high for nearly the entire execution after.

To improve our speedup, we need to modify the accelerator to prevent the vector unit from stealing too much bandwidth from the systolic array. We tried multiple approaches, including modifying the job scheduler and altering the clock rate of the vector unit to reduce its memory intensity. However, each of these had unintended side effects that either didn’t improve our result or resulted in a net slowdown. One solution that did work was changing the memory controller to prioritize DRAM transactions from the systolic array over those from the vector unit. Since the premise of overlapping these operations was that softmax would steal unused memory during the systolic array computation, we believed this made sense. This reduced the total runtime for this workload from 8.77ms to 8.57ms, close to our ideal speedup.

Knowing *why* performance increases or decreases as a result of a change is as important as knowing that it changed at all. While high-level dataflow mapping techniques are highly productive for homogeneous accelerators, currently available techniques cannot or do not consider parallel model execution or heterogeneous computation. Cycle-accurate simulation models make these takeaways possible, and we have shown how COCOSSim can provide users with actionable implementation details and that COCOSSim provides an intuitive yet accurate model for identifying bottlenecks and experimenting with architectural and microarchitectural improvements to heterogeneous machine learning accelerators.

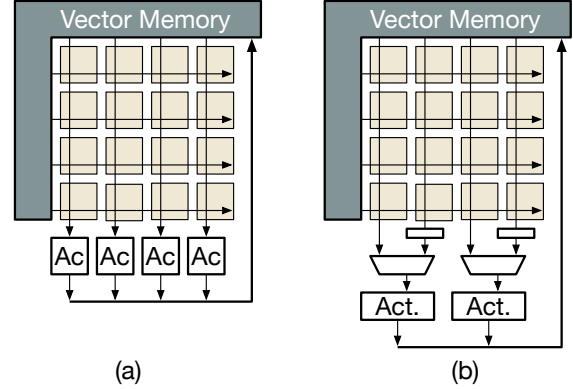


Fig. 8: Activation units can be tightly fused to the outputs of the systolic array. We assume that the outputs can be written to buffer without additional overheads when not using the activation units and that each unit is fully pipelined. (a) One activation unit per output will be able to sustain throughput to the vector memory but will incur unnecessary memory overhead. (b) When the latency of the FPU in the systolic array processing element is greater than one, we can re-use the activation unit hardware for multiple systolic array outputs, saving hardware.

B. Operation Fusion

An increasingly popular approach for specialized machine learning accelerators is operation-fusion [14]. Operation fusion (op-fusion) “fuses” sets of consecutive operations together into a single, optimized primitive. A common example is the floating-point Fused-Multiply-Add (FMA) available in most floating-point IPs. Op-fusion can increase throughput, decrease pipeline depth, and reduce area, but it can also impact an accelerator’s flexibility and require additional fallback paths for inputs that cannot use the fused operation. In machine learning, the most common target for operation fusion by far is the softmax activation inside the attention mechanism [9], [23], [29]. Still, other activation functions have been accelerated in this way as well [23].

We can analyze the feasibility of adding fused activation units to our TPU-like systolic array using COCOSSim by assessing the costs of activation functions in the vector unit and modifying the systolic array state transition matrix discussed earlier in Table I.

We estimate the potential speedup by considering a $4096 \times 320 \times 320$ matrix multiply followed by a SiLU activation. The matrix multiply takes $105\mu\text{s}$, and the activation execution takes $22\mu\text{s}$, leaving an opportunity for a 17% speedup by fusing the activation execution into the systolic array machine.

Activation units can be fused in multiple locations. One option is placing a vector of activation units by the write port, using the high W_{next} penalty from the $3 \rightarrow 4$ transition to hide any additional C_{next} penalty from computing activations. This will not induce additional latency so long as we can compute activations as fast as we can stream to memory. This is 64

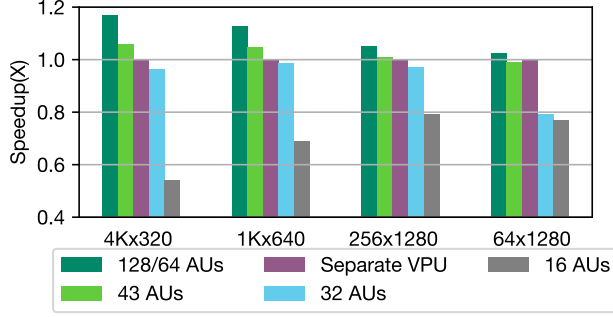


Fig. 9: We measure the performance of a SiLU activation unit fused with a 128x128 systolic array against the baseline systolic array and independent vector unit combination. The x-axis denotes the size of the output activation matrix. Sharing activation units between columns of the systolic array allows greater resource sharing, but reduces throughput. Using 128 or 64 Activation Units (AUs) achieves the ideal speedup, but we also find that using fewer activation units and increasing systolic array drain penalties can still result in speedups.

bytes per HBM channel per cycle, resulting in a throughput of 256 operands per cycle for our analysis. Such a high activation throughput would incur significant area overheads for a 17% speedup. Luckily, there is an alternative.

Our second option is computing activations as inputs are transferred from the systolic array to the buffer, as shown in Figure 8(a). This modification would modify the C_{next} parameter for all transitions into states 2 or 3, where inputs are streamed through/out of the systolic array. Currently, we incur a minimum cycle count of $C_{\text{next}} = \max(l_{\text{fpu}}, B) * M$ when transitioning into state 2 because the result of the previous operation’s accumulation must be complete before it is passed to the neighboring processing element. This results in a peak throughput of $1/\max(l_{\text{fpu}}, B)$. Any activation units added to the systolic array’s outputs will only need to match the throughput of the systolic array FPU’s. This results in a modified minimum cycle requirement for state 2: $C'_{\text{next}} = \max(t_{\text{act}}, \max(l_{\text{fpu}}, B)) * M + l_{\text{act}}$, for activation unit throughput/latency $t_{\text{act}}/l_{\text{act}}$.

Observe that if the activation unit is fully pipelined ($t_{\text{act}} = 1$), then the state machine is unchanged except for the minor additional latency, l_{act} . Further, we experimentally found the greatest alignment to TPUv3 performance results for $l_{\text{fpu}} = 2$, providing an opportunity for resource sharing between systolic array output columns.

We can take advantage of this by reducing the number of activation units and sharing the units between multiple systolic array outputs, as shown in Figure 8(b). With this optimization in mind, we can further parameterize our state machine as a function of the number of activations (n_{act}).

$$C_{\text{next}}^{(2)} = \max\left(\frac{s * t_{\text{act}}}{n_{\text{act}}}, \max(l_{\text{fpu}}, B)\right) * M + l_{\text{act}}$$

We ran this modified model for various matrix sizes to determine the feasibility of this proposed optimization. COCOSSim reports, as we suspected, that there is no additional benefit to having one activation unit per systolic array output and that only s/l_{fpu} activations are necessary to achieve peak performance. We show the results for fewer activation units in Figure 9, and, surprisingly, we can glean moderate performance benefits for even as few as 43 activation units.

While operation fusion for activation may not be a fruitful addition in the presence of an on-chip 1024-wide vector unit, these results show that implementations that do not need a vector unit for other purposes (e.g., softmax) can reasonably pursue operation fusion as an area-efficient way to perform activation without a wide on-chip vector unit. This case study shows that COCOSSim enables us to propose, analyze, and measure the impact of significant architectural and microarchitectural changes.

VI. RELATED WORK

Existing simulators such as MAESTRO [15], Timeloop [18], STONNE [17], SCALE-Sim [26] and TPTUSim [5] have been instrumental in advancing the development and optimization of conventional deep learning accelerators. Despite their utility, these simulators cannot often model the increasingly complex architectures required by modern workloads, such as heterogeneous systems, multi-core scalability, and non-linear operations support. A comparative overview of these simulators and our framework is presented in Table IV.

MAESTRO [15] is a high-level framework that models CNN accelerators using flexible, software-pragma-like dataflow descriptors. This approach allows users to specify hardware architectures with minimal effort and evaluate performance trade-offs in data movement, energy consumption, and computational throughput. However, this high-level abstraction makes it challenging to model low-level architectural details, such as memory hierarchy interactions and operation-specific latencies. It also lacks support for multi-core or heterogeneous architectures, limiting its applicability to modern designs.

Timeloop [18] and SCALE-Sim [26] provide micro-architecture-aware simulation frameworks that enable more detailed modeling of CNN accelerator components. However, these architectures are primarily focused on dataflow mapping and do not accurately model DRAM, a significant performance bottleneck for many neural network workloads.

STONNE [17] is a microarchitecture-focused cycle-accurate simulator tool for neural network engines, offering high accuracy on account of offering building blocks that are closely coupled to the intended hardware implementation (e.g., wires, adders, multipliers). It currently supports a variety of reduction-tree dataflows and an output-stationary systolic array. Building such a model is straightforward when the hardware implementation is well understood. The resulting model can achieve fantastic accuracy because it can be verified against an existing RTL implementation. However, low-level

Modeling Framework	μ Arch Flexibility	Support of Multi-Core	Cycle Accurate	Model Speed
MAESTRO [15]	Low	No	No	High
TPTPUSim [5]	Low	No	Yes	Med
Timeloop [18]	Med	No	No	Med
STONNE [17]	Med	No	Yes	Low
SCALE-Sim [26]	Med	No	Yes	High
COCOSSim	High	Yes	Yes	High

TABLE IV: Comparison of COCOSSim’s capabilities to other simulators.

specifications can come with costs as well. For instance, it can be difficult to model an architecture whose implementation is mostly unknown, and modifying existing architectures for new dataflows (e.g., weight-stationary systolic arrays) can be cumbersome. There are also significant computational costs to such low-level modeling, decreasing simulation speed.

TPTPUSim [5] is specifically tailored to model the TPU v1 by including various TPU-specific latencies and costs. While TPTPUSim uses cycle-accurate DRAM simulation and provides reasonable accuracy for TPU v1, its lack of support for multi-core designs and limited flexibility for architectural modifications hampers its adaptation to newer TPU generations. These constraints make it challenging to explore design variations or broader architectural innovations.

To our knowledge, COCOSSim is the first simulator that can consider heterogeneous mixes of systolic arrays and supporting units or the presence and interaction of multiple concurrent compute units.

VII. CONCLUSION

The rapid advancements in neural network workloads have introduced the need for more sophisticated and adaptable performance simulators capable of modeling heterogeneous architectures and modern computing paradigms. Existing frameworks are limited in their ability to model critical aspects such as memory latencies, vector units for non-linear operations, and multi-core architectures.

To address these challenges, we present COCOSSim, a flexible, cycle-accurate simulation framework that bridges high-level modeling flexibility and low-level architectural detail. Unlike prior methodologies, COCOSSim supports architectural, microarchitecture, memory controller, and scheduling modifications, enabling detailed performance evaluations for diverse workloads.

Through comprehensive evaluation, we demonstrate that COCOSSim achieves high modeling accuracy and simulation efficiency, paving the way for its adoption in future accelerator research and development.

VIII. ACKNOWLEDGMENT

This project is supported in part by a National Science Foundation CAREER award, CCF-2045973, and in part by a National Science Foundation Athena AI Institute award, CNS-2112562.

REFERENCES

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.
- [2] Alphabet. Edge tpu. <https://cloud.google.com/edge-tpu>, March 2023.
- [3] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarakar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. PyTorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 929–947. ACM, 2024.
- [4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.
- [5] Jinha Chung. Tptpu-sim: A toy-purpose tpu simulator, <https://github.com/jinhachung/tptpu-sim.git>.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2019.
- [7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2021.
- [8] Google LLC. Cloud tensor processing units (TPUs). <https://cloud.google.com/tpu/docs/tpus>, July 2020.
- [9] Tae Jun Ham, Sung Jun Jung, Seonghak Kim, Young H. Oh, Yeonhong Park, Yoonho Song, Jung-Hun Park, Sanghee Lee, Kyoung Park, Jae W. Lee, and Deog-Kyoon Jeong. A³: Accelerating attention mechanisms in neural networks with approximation. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 328–341, 2020.
- [10] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *The Computing Research Repository (CoRR)*, abs/2006.11239, 2020.
- [11] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, Xiang Zhou, Zongwei Zhou, and David A Patterson. TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, June 2023.
- [12] Norman Jouppi, Doe Yoon, George Kurian, Sheng Li, Nishant Patil, James Laudon, Cliff Young, and David Patterson. A domain-specific supercomputer for training deep neural networks. *Communications of the ACM*, 63:67–78, 06 2020.
- [13] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb,

- Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, aron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, and Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omerick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiana, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2017.
- [14] Sehoon Kim, Coleman Hooper, Thanakul Wattanawong, Minwoo Kang, Ruohan Yan, Hasan Genc, Grace Dinh, Qijing Huang, Kurt Keutzer, Michael W. Mahoney, Yakun Sophia Shao, and Amir Gholami. Full stack optimization of transformer inference: a survey. *arXiv preprint arXiv:2302.14017*, 2023.
- [15] Hyoukjun Kwon, Prasanth Chatarasi, Michael Pellauer, Angshuman Parashar, Vivek Sarkar, and Tushar Krishna. Understanding reuse, performance, and hardware cost of DNN dataflows: A data-centric approach. In *Proceedings of the IEEE International Symposium on Microarchitecture (MICRO)*, October 2019.
- [16] Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce Jacob. Dramsim3: A cycle-accurate, thermal-capable dram simulator. *IEEE Computer Architecture Letters*, 19(2):106–109, 2020.
- [17] Francisco Muñoz-Martínez, José L. Abellán, Manuel E. Acacio, and Tushar Krishna. Stonne: Enabling cycle-level microarchitectural simulation for dnn inference accelerators. In *Proceedings of the International Symposium on Workload Characterization (IISWC)*, 2021.
- [18] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Bruce Khailany, Stephen W. Keckler, and Joel Emer. Timeloop: A systematic approach to dnn accelerator evaluation. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 304–315, 2019.
- [19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. *arXiv preprint arXiv:1912.01703*, 2019.
- [20] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. Zero-shot text-to-image generation. *The Computing Research Repository (CoRR)*, abs/2102.12092, 2021.
- [21] James K. Reed, Zachary DeVito, Horace He, Ansley Ussery, and Jason Ansel. Torch.fx: Practical program capture and transformation for deep learning in python. *arXiv preprint arXiv:22112.08429*, 2022.
- [22] Isakado/DNNsim repository. <https://github.com/isakado/dnnsim.git>.
- [23] Eyes Robson, Ceyu Xu, and Lisa Wu Wills. Prose: the architecture and design of a protein discovery engine. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, page 655–668, New York, NY, USA, 2022. Association for Computing Machinery.
- [24] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. *arXiv preprint arXiv:2112.10752*, 2021.
- [25] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily Denton, Seyed Kamyar Seyed Ghasemipour, Burcu Karagol Ayan, S. Sara Mahdavi, Rapha Gontijo Lopes, Tim Salimans, Jonathan Ho, David J Fleet, and Mohammad Norouzi. Photorealistic text-to-image diffusion models with deep language understanding, 2022.
- [26] Ananda Samajdar, Jan Moritz Joseph, Yuhao Zhu, Paul Whatmough, Matthew Mattina, and Tushar Krishna. A systematic methodology for characterizing scalability of dnn accelerators using scale-sim. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 58–68, 2020.
- [27] Hugo Touvron and et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2023.
- [29] Hanrui Wang, Zhekai Zhang, and Song Han. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 97–110, 2021.
- [30] Yannan Nellie Wu, Joel S Emer, and Vivienne Sze. Accelerger: An architecture-level energy estimation methodology for accelerator designs. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2019.

APPENDIX A

ARTIFACT APPENDIX

A. Abstract

This artifact appendix describes how to run the COCOSSim simulator, a cycle-accurate performance model designed to evaluate deep learning workloads on hardware accelerators. The simulator models various linear and non-linear neural network operations, including Matmul, Conv, Softmax, Attention, and ResNet. Users can obtain performance metrics such as cycle counts, latencies, and other utilization under different architectural and memory configurations.

The artifact includes the full implementation of the simulator, setup instructions, and scripts for running a series of experiments. These experiments reproduce the results presented in the paper, allowing others to replicate the experiments and verify the reported findings.

B. Artifact check-list (meta-information)

- **Program:** ISPASS 2025
- **Compilation:** GCC
- **Runtime Environment:** C++, Python 3.6+
- **Binary:** ./perf_model
- **Hardware:** CPU
- **Metrics:** Cycle count, Latency
- **Output:** ML Workload Performance
- **Experiments:** Figure 6
- **How much disk space required (approximately)?:** 33 MB
- **How much time is needed to prepare workflow (approximately)?:** 15 mins
- **How much time is needed to complete experiments (approximately)?:** 15 mins
- **Publicly available?:** Yes

C. Description

COCOSSim is a cycle-accurate simulator designed to evaluate the performance of modern deep-learning workloads on hardware accelerators. The simulator integrates systolic arrays, vector units, and realistic memory models to provide an in-depth analysis of the performance of deep learning model operations like Matmul, Conv2D and Softmax across different configurations. For the experiments, the simulator is configured to TPU v3 hardware configuration to verify the results in Figure 6.

1) How to access

The zip of a version of the COCOSSim code that reproduces the paper results is available at: <https://doi.org/10.6084/m9.figshare.28614851>.

2) Hardware dependencies

- CPU
- 4MB of disk space

3) Software dependencies

- Operating System: Any OS (Linux, macOS, or Windows with WSL are supported).
- CMake: Version 3.10 or higher is required for building the project.
- Libraries: Standard system libraries like `glibc` and `gcc`, as well as common Python libraries such as `matplotlib` and `numpy`, are required for compilation..

D. Installation and Experiment Workflow

The instructions to install the dependencies and build the simulator are supplied in the README of the code repository. In brief, the workflow consists of the following steps:

- 1) Clone the repository and initialize DRAMSim3 using the provided setup script:

```
$ git clone <repo-url>
$ cd <repo-directory>
$ bash setup_dramsim.sh
```

- 2) Build the simulator:

```
$ mkdir build
$ cd build
$ cmake .. -DUSE_VCD=ON
$ make
```

- 3) Run the evaluation script:

```
$ cd ae_plots
$ python experiments.py
```

- 4) Generate plots:

```
$ python plot_parse.py
```

This runs the simulator over a set of predefined workloads, writes results to a file, and optionally visualizes them using the supplied plotting script.

E. Evaluation and expected results

The provided code will produce the COCOSSim values shown in Figure 6 of the paper. Each operation is simulated on a TPU v3-like configuration of the COCOSSim simulator. The expected output includes total cycle counts for individual operations and end-to-end layer sequences (e.g., ResNet blocks). Reference outputs from the TPU v3 backend are included to be used by the plotting script to reproduce the results.